

Deep Learning Lab 2

Suraj Karki

Department of Engineering Science
University West
Trollhättan, Sweden
suraj.karki@student.hv.se

Nils Karlsson

Department of Engineering Science
University West
Trollhättan, Sweden
nils.Karlsson@student.hv.se

Nils Wikström

Department of Engineering Science
University West
Trollhättan, Sweden
nils.wikstrom@student.hv.se

Index Terms—Deep Learning, Laboration 2, PCA analysis, Dimensionality Reduction

Abstract—The source code for this lab can be found at [1]. This lab focuses on Principal Component Analysis (PCA) and Kernel PCA for dimensionality reduction. We load the Iris dataset, implement PCA both manually and using sklearn, and train neural networks on the original and reduced datasets. We also explore Kernel PCA with different kernels and build a pipeline to tune hyperparameters for both Kernel PCA and the neural network.

I. TASK 1: PRINCIPAL COMPONENT ANALYSIS (PCA)

A. 1- Load or create a dataset with more than 2 dimensions.

Data is loaded from sklearn datasets library. Iris dataset is used for this lab. It has 4 features (dimensions) and 3 classes.

B. 2- Find the first 2 principal components with and without using sklearn.

using sklearn, PCA from sklearn.decomposition is used to find the first 2 principal components. Without sklearn, PCA is implemented using numpy as shown below:

Figure 1 shows the manual PCA implementation using numpy.

C. 3- Practice using PCA to preserve a certain percentage of variance.

Using sklearn PCA, we can preserve a certain percentage of variance by setting the `n_components` parameter to a float value between 0 and 1. For example, to preserve 95% of variance, we can set `n_components=0.95`.

D. 4- Train a classification or regression neural network by using: a. The Original dataset b. Principal components

Using `MLPClassifier` from `sklearn.neural_network`, we trained a neural network on both the original dataset and the principal components. Performance was evaluated using the accuracy score from `sklearn.metrics`. With the original data, an accuracy of 93% was achieved, while PCA-reduced data yielded 97%. The manual PCA implementation achieved an accuracy of 96.6%.

E. 5- Repeat part 4 using Kernel PCA (linear, sigmoid, RBF).

Using `KernelPCA` from `sklearn.decomposition`, we performed Kernel PCA with different kernels. For example, to use linear kernel, we can set `kernel='linear'`. The performance is compared using accuracy score from `sklearn.metrics`. With linear kernel 97% accuracy is achieved, sigmoid kernel yields only 36% accuracy and RBF kernel gives 96.6% accuracy. This shows that sigmoid activation does not perform well with this nearly linearly separable dataset. It saturates easily and fails to learn complex patterns. It also overlaps the classes in the feature space. Figure 2 shows the PCA results using sigmoid kernel.

Figure 3 shows the PCA result using linear kernel on Iris dataset. Linear kernel and RBF kernel performs well on this dataset as it is nearly linearly separable.

F. 6- Build a pipeline to tune the hyperparameters of Kernel PCA and also the neural network. Which hyperparameters can be tuned?

Using `Pipeline` and `GridSearchCV` from `sklearn.model_selection`, we built a pipeline to tune the hyperparameters of Kernel PCA and the `MLPClassifier`. The hyperparameters that can be tuned for kernel PCA are:

- number of components in Kernel PCA
- Kernel type (linear, sigmoid, RBF)
- gamma value for RBF and sigmoid kernels
- `coef0` value for sigmoid kernel

For `MLPClassifier`, the hyperparameters that can be tuned are:

- Neural network solver (adam, sgd, lbfgs)
- Number of components
- Neural network hidden layer sizes
- Neural network activation function
- Neural network learning rate
- Neural network max iterations
- Neural network regularization parameter (alpha)

II. CLASSIFICATION PROBLEM

III. REGRESSION PROBLEM

REFERENCES

- [1] GitHubRepo, "Authors github repository for laborations," Online: <https://github.com/Nilswik/DIL700/>, accessed: 2025-09-24.

```

# PCA without using sklearn
def pca_manual(X, n_components):
    # Center the data
    X_centered = X - np.mean(X, axis=0)

    # Compute covariance matrix
    covariance_matrix = np.cov(X_centered, rowvar=False) # sklearn uses rowvar=False by default

    # Eigen decomposition
    eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix) # sklearn uses svd decomposition but eigh is fine here for covariance matrix

    # Sort eigenvalues and eigenvectors
    sorted_indices = np.argsort(eigenvalues)[::-1] # Descending order here, sklearn order by variance
    sorted_eigenvectors = eigenvectors[:, sorted_indices]

    # Select top n_components
    selected_eigenvectors = sorted_eigenvectors[:, :n_components]

    # Transform data
    X_reduced = np.dot(X_centered, selected_eigenvectors) # equivalent to pca.fit_transform(X) in sklearn

    return X_reduced

```

Fig. 1: Manual PCA implementation using numpy.

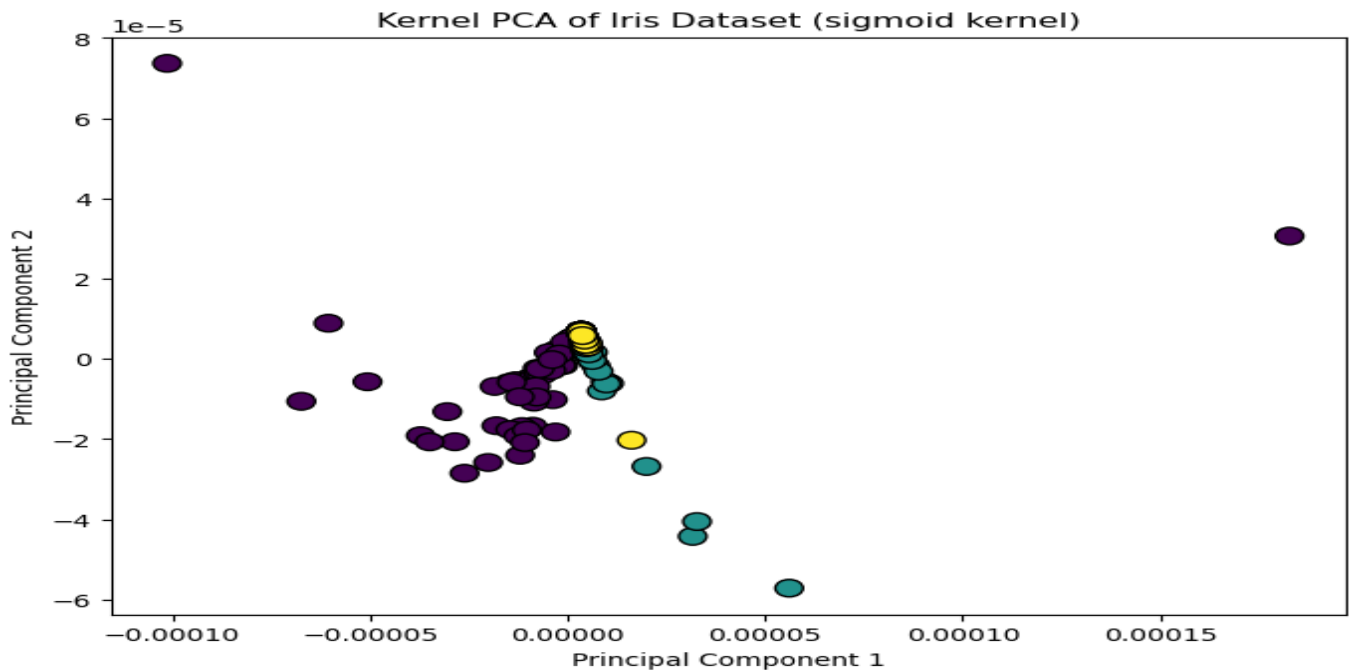


Fig. 2: Demonstrates the poor performance of sigmoid kernel in PCA.

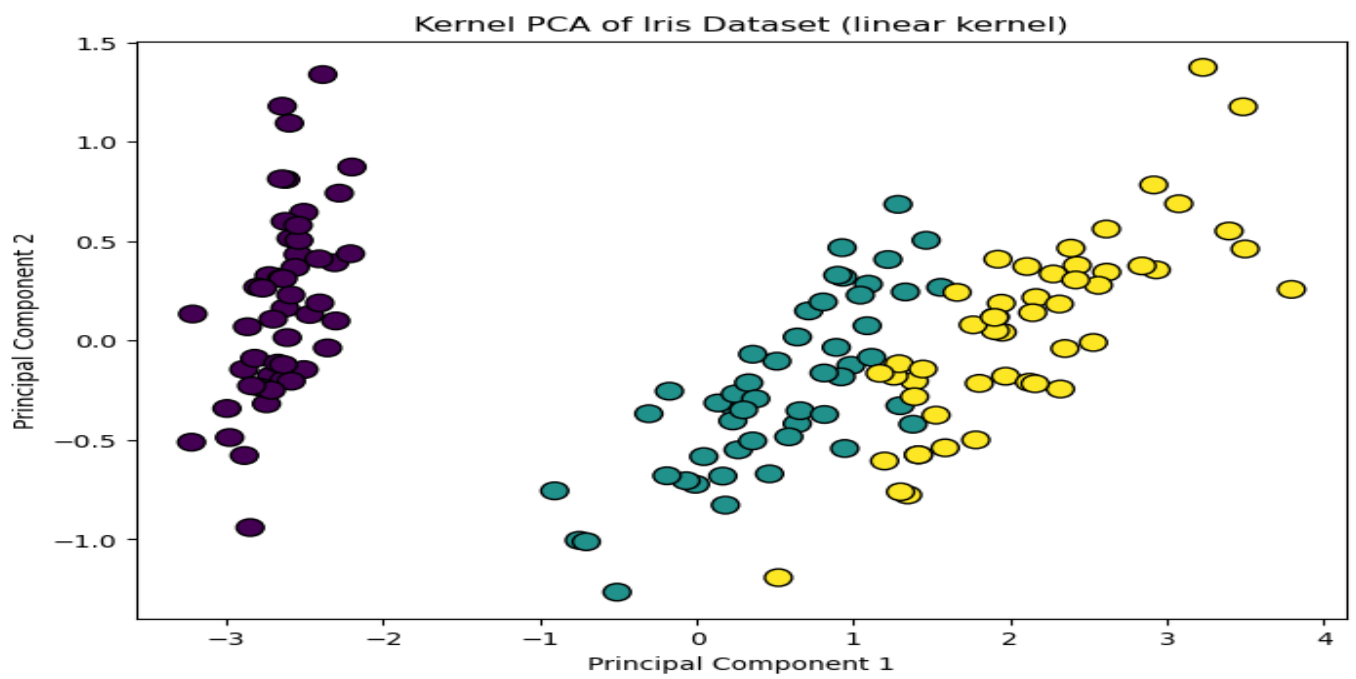


Fig. 3: PCA using linear kernel on Iris dataset (demonstration of kernel PCA).